# AWS VPC Peering Project

Connecting two VPCs and proving the traffic stays private

| | |
|---|---|
| **Candidate** | Arun Bharathi M B |
| **Program** | AWS Architect Associate / Cloud Systems Engineering |
| **Project** | Connect Two VPCs & Prove Private Connectivity (VPC Peering) |
| **Region** | Asia Pacific (Sydney), ap-southeast-2 |
| **Completed** | October 2026 |
| **Deliverable** | Final project implementation and technical defense report |

### Contents

1. Overview
2. Background Concepts
3. Architecture and Design
4. Resources, Address Plan and Firewall Rules
5. Step-by-Step Build (with Screenshots)
6. Testing and Results
7. Fault Injection Experiment
8. Troubleshooting Guide
9. Security Considerations, Limitations and Improvements
10. Viva Questions and Answers
11. Cost Control and Cleanup
12. Conclusion and Lessons Learned
Appendix A: Equivalent AWS CLI Commands
Appendix B: SSH Configuration for ProxyJump
Appendix C: Glossary
Appendix D: Final Verification Checklist

## 1. Overview

Backend servers such as app runtimes, databases and queues do not need to be reachable from the public internet, and exposing them only widens the attack surface. The usual practice is to keep them in private subnets with private IPv4 addresses only, and to open one controlled way in.

In this project I built two separate VPCs and linked them with a VPC peering connection. VPC A stands in for the front-end / admin side and VPC B for the private backend. Traffic between them travels over AWS's own network, so it never touches the internet, and I did not need a VPN or a NAT gateway for it.

### 1.1 What I set out to do

- Build two VPCs with address ranges that do not overlap: `10.10.0.0/16` (VPC A) and `10.20.0.0/16` (VPC B).
- Put a public and a private subnet in VPC A, and one private subnet in VPC B, each tied to its own route table.
- Use a single bastion host in VPC A, together with OpenSSH ProxyJump, as the only way to administer the private instances.
- Bring up a peering connection, `pcx-0273ef24e21641107`, and add routes on both sides of it.
- Prove it works, and prove what happens when it does not: ping before peering (100% loss), after peering (0% loss), and again after deliberately removing a security group rule.

### 1.2 Scope

Everything was built in the Sydney region (ap-southeast-2) using IPv4 only. The project covers private connectivity between the two VPCs and administrative access through the bastion. It does not cover production extras such as a NAT gateway, VPC flow logs, custom network ACLs or high availability across several availability zones. I list these in section 9 as the next improvements, because a real deployment would need them.

### 1.3 How to read this report

Section 2 explains the networking ideas the project relies on, section 3 shows the design, and sections 4 and 5 list and walk through what I built, with screenshots. Sections 6 and 7 show the test results. Sections 8 to 12 cover troubleshooting, security, viva answers, cost and what I learned. The appendices contain the equivalent AWS CLI commands, an SSH configuration example, a glossary and a final checklist.

## 2. Background Concepts

Before the build, here is a short refresher on the pieces this project uses. I included it so the later sections are easier to follow and so I could explain each design choice in the viva.

### 2.1 VPCs and CIDR blocks

A Virtual Private Cloud (VPC) is a logically isolated network inside one AWS region. When you create one you pick an IPv4 range in CIDR notation. A `/16` block such as `10.10.0.0/16` contains 65,536 addresses. The number after the slash says how many leading bits are fixed, so a smaller number means a bigger network.

Peering has one hard rule: the two VPCs must not have overlapping ranges. If both used `10.0.0.0/16`, a router could not tell which VPC an address like `10.0.1.5` belongs to. That is why I picked `10.10.0.0/16` and `10.20.0.0/16`, which are completely separate.

### 2.2 Subnets: "public" and "private"

A subnet is a slice of the VPC range that lives in exactly one availability zone. Each of my subnets is a `/24`, which gives 256 addresses, of which 251 are usable because AWS reserves five in every subnet (the network address, the VPC router, the DNS resolver, one for future use and the broadcast address). Choosing fixed private IPs such as `10.10.1.10` was safe because the host part was well above that reserved range.

Nothing about a subnet is public or private by itself. What makes a subnet "public" is its route table: it has a route to an internet gateway, and the instances in it have public IPs. A "private" subnet has no such route. In this project `ewb-a-public` is the only public subnet.

### 2.3 Route tables and the internet gateway

A route table is a list of destination ranges and targets. Every table has an automatic **local** route that lets instances inside the same VPC talk to each other. When a packet leaves an instance, the router picks the most specific matching route (the longest prefix). So a packet to `10.20.1.10` skips the local route for `10.10.0.0/16` and uses a `10.20.0.0/16` route instead, if one exists.

An internet gateway (IGW) is the door between a VPC and the internet. Only subnets whose route table sends `0.0.0.0/0` to the IGW can use it. I attached an IGW to VPC A only, and none to VPC B.

### 2.4 Security groups and network ACLs

These are the two firewall layers in a VPC, and the project depends on both being understood. The table compares them.

| | Security group | Network ACL |
|---|---|---|
| Attached to | Individual network interface (instance) | Whole subnet |
| State | Stateful: replies to allowed traffic are let back automatically | Stateless: inbound and outbound rules are checked separately |
| Rule types | Allow rules only | Allow and deny rules, evaluated in numbered order |
| Default for a new one | No inbound allowed, all outbound allowed | The default ACL that comes with a VPC allows everything |
| Used in this project | Yes, with single /32 sources (section 4.2) | Left as the VPC default, so it does not block traffic |

### 2.5 VPC peering

A peering connection is a one-to-one networking link between two VPCs. Once it is accepted and routes are added, instances in either VPC can reach each other using private IPs. AWS runs it on its own infrastructure, so there is no gateway, VPN appliance or single choke point to manage. A few properties matter for this project:

- **Both sides must agree.** One VPC (the requester) asks, and the other (the accepter) must accept. Mine went from pending to Active after acceptance.
- **It adds no routes by itself.** You have to add a route in each subnet route table that should use the link.
- **It is non-transitive.** If A is peered with B and B with C, A still cannot reach C through B. It also means B cannot borrow A's internet gateway.
- **Ranges cannot overlap**, as explained above.
- **Security groups and ACLs still apply.** Peering opens a path, not permission.

### 2.6 Bastion host and SSH ProxyJump

A bastion host (or jump host) is a small, hardened server that sits in a public subnet and is the single entry point for administration. The private instances never get public IPs, and their security groups only trust the bastion.

OpenSSH's `ProxyJump` option makes this convenient. When I run `ssh ewb-app-b`, my laptop first connects to the bastion, and the bastion then opens a second connection to the target. The two legs are chained so it feels like a single login, and no private key needs to be copied onto the bastion. The configuration is shown in Appendix B.

## 3. Architecture and Design

The diagram below shows the finished layout: my laptop reaches the bastion through the internet gateway, and everything else is private. The green arrow is the ping test and the orange dashed lines are the administrative SSH paths.

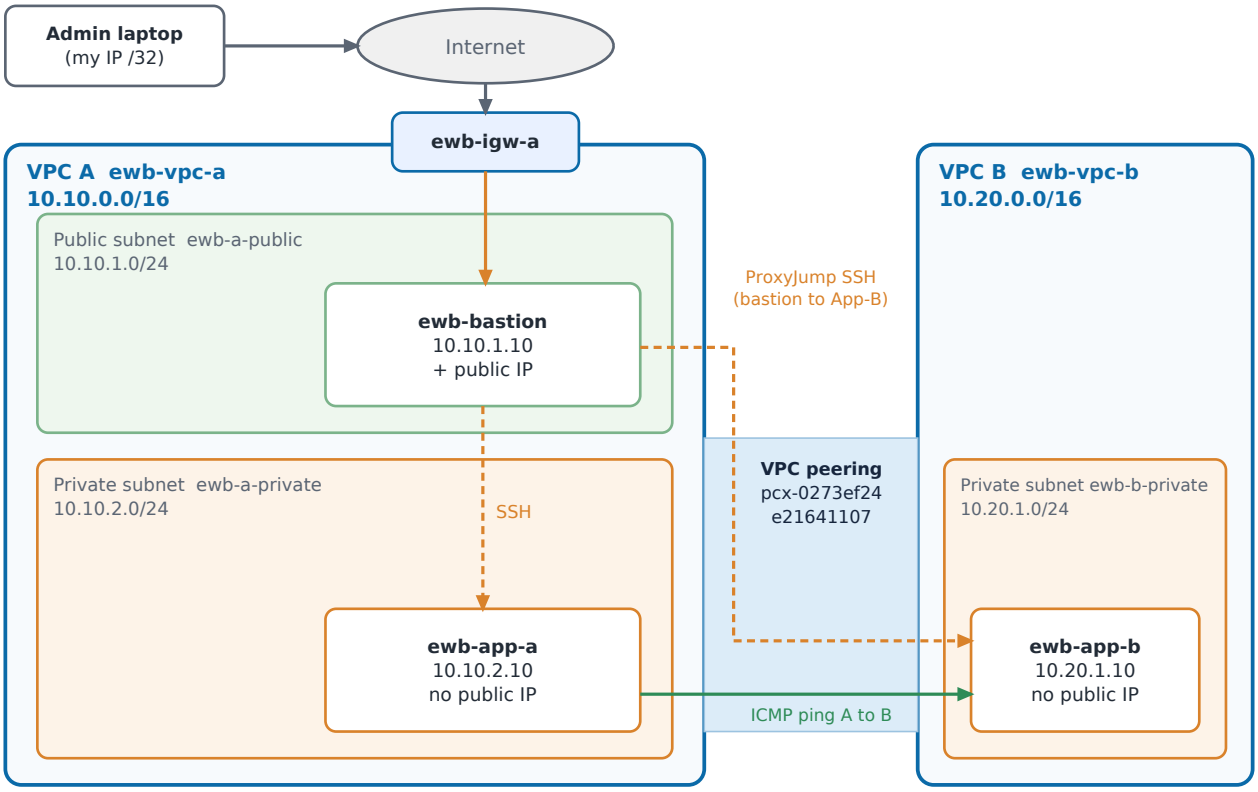



*Figure 3.1: Architecture of the two VPCs, the peering connection and the bastion access path.*

## 3.1 Design decisions

The table lists the main choices and the reason behind each one.

| Decision | What I did | Why |
|---|---|---|
| Address ranges | 10.10.0.0/16 and 10.20.0.0/16 | They do not overlap, which peering requires, and they leave room for more subnets later. |
| VPC B has no IGW | No internet gateway attached | Nothing in VPC B can be reached from, or send traffic directly to, the internet. |
| One bastion | A single host in the public subnet of VPC A | One controlled entry point is easier to lock down and audit than many. |
| No public IPs on apps | Both app instances are private only | They cannot be scanned or attacked from the internet. |
| Fixed private IPs | 10.10.1.10, 10.10.2.10, 10.20.1.10 | Security group rules use /32 addresses, so the addresses must not change. |
| /32 firewall sources | Single-host sources only | Least privilege: only the exact machine that needs access gets it. |
| Explicit route tables | Custom table per subnet instead of the main table | Each subnet's routing is visible and cannot change by accident. |
| Return route in VPC B | 10.10.0.0/16 to the peering ID | Replies need a path back; routing is decided per direction. |

## 3.2 How a ping travels from App-A to App-B

Tracing one packet through every check is the best way to see why the project needs routes, security groups and the peering link all at once. This is the path of one echo request and its reply, assuming the default network ACLs and the default allow-all outbound security group rule:

| Step | What happens | Which setting decides it |
|---|---|---|
| 1 | App-A (10.10.2.10) sends an echo request to 10.20.1.10. | Destination is outside 10.10.0.0/16, so it goes to the subnet router. |

| Step | What happens | Which setting decides it |
|---|---|---|
| 2 | Outbound security group check on App-A. | AWS creates security groups with allow-all outbound, so it passes. |
| 3 | The router looks up ewb-rt-a-private. | 10.20.0.0/16 matches the peering route (more specific than 0.0.0.0/0, which this table does not have). |
| 4 | The packet crosses to VPC B. | The peering connection must be Active. |
| 5 | Inbound checks at App-B. | Network ACL allows it; ewb-sg-app-b must allow ICMP echo from 10.10.2.10/32. |
| 6 | App-B builds the echo reply to 10.10.2.10. | Security groups are stateful, so the reply is allowed automatically. |
| 7 | The VPC B router looks up ewb-rt-b-private. | 10.10.0.0/16 must point at the peering ID, or the reply is dropped. |
| 8 | The reply reaches App-A and ping prints a time. | Round trip averaged 0.776 ms in my test. |

> **Takeaway**
> If any single step above is missing, the symptom is the same: ping just times out. That is why the tests in section 6 and the fault injection in section 7 each break exactly one step and watch what changes.

# 4. Resources, Address Plan and Firewall Rules

## 4.1 Resource inventory

Everything I created, with its ID and where it sits:

| Name | Role | Resource ID | CIDR / IP | Placement |
|---|---|---|---|---|
| ewb-vpc-a | VPC A (requester) | `vpc-0dc09873972f21f1` | 10.10.0.0/16 | ap-southeast-2 |
| ewb-a-public | Public subnet A | `subnet-01c495c71b5169e34` | 10.10.1.0/24 | ap-southeast-2a |
| ewb-a-private | Private subnet A | `subnet-07aaa750fda8206d0` | 10.10.2.0/24 | ap-southeast-2a |
| ewb-igw-a | Internet gateway | `igw-0cbffc1dd4299a806` | n/a | Attached to VPC A |
| ewb-vpc-b | VPC B (accepter) | `vpc-01a6d0244af8f29c2` | 10.20.0.0/16 | ap-southeast-2 |
| ewb-b-private | Private subnet B | `subnet-09dada267bb3eefb9` | 10.20.1.0/24 | ap-southeast-2b |
| ewb-rt-a-public | Public route table | `rtb-0cda91d276c8336ef` | 0.0.0.0/0 to IGW; 10.20.0.0/16 to peering | Linked to ewb-a-public |
| ewb-rt-a-private | Private route table A | `rtb-0d6d126749f956d08` | 10.20.0.0/16 to peering | Linked to ewb-a-private |
| ewb-rt-b-private | Private route table B | `rtb-009768c6a16c13c15` | 10.10.0.0/16 to peering | Linked to ewb-b-private |
| ewb-a-b-peer | VPC peering connection | `pcx-0273ef24e21641107` | n/a | Active, links VPC A and B |
| ewb-bastion | Bastion host (EC2) | `i-077192b847673b898` | 10.10.1.10 (+ public IP) | ewb-a-public |
| ewb-app-a | Private app A (EC2) | `i-07b73257444e21c88` | 10.10.2.10 (no public IP) | ewb-a-private |
| ewb-app-b | Private app B (EC2) | `i-08b09b84e5873c96d` | 10.20.1.10 (no public IP) | ewb-b-private |

## 4.2 Security group rules matrix

Every rule uses a single /32 address as its source. The bastion trusts only my own administrator IP, and the private instances trust only the bastion and their peer instance.

| Security group | Protocol / port | Source | Purpose |
|---|---|---|---|
| ewb-sg-bastion | TCP 22 (SSH) | Administrator IP /32 | Lets me log in to the bastion and nobody else. |
| ewb-sg-app-a | TCP 22 (SSH) | 10.10.1.10/32 (bastion) | Administration through the bastion only. |
| ewb-sg-app-a | ICMP echo request | 10.20.1.10/32 (App-B) | Allows the reverse ping from the peer instance. |

| Security group | Protocol / port | Source | Purpose |
|---|---|---|---|
| ewb-sg-app-b | TCP 22 (SSH) | 10.10.1.10/32 (bastion) | Administration through the bastion only. |
| ewb-sg-app-b | ICMP echo request | 10.10.2.10/32 (App-A) | Allows the forward ping. This is the rule I removed in section 7. |

Outbound rules were left at the AWS default of allow-all. Because security groups are stateful, replies to allowed inbound traffic need no extra rule.

# 5. Step-by-Step Build (with Screenshots)

## 5.1 VPC A and its subnets

I worked in the Sydney region (ap-southeast-2). VPC A was created with the CIDR `10.10.0.0/16`. Inside it, in availability zone ap-southeast-2a, I added a public subnet `ewb-a-public` (10.10.1.0/24) and a private subnet `ewb-a-private` (10.10.2.0/24).

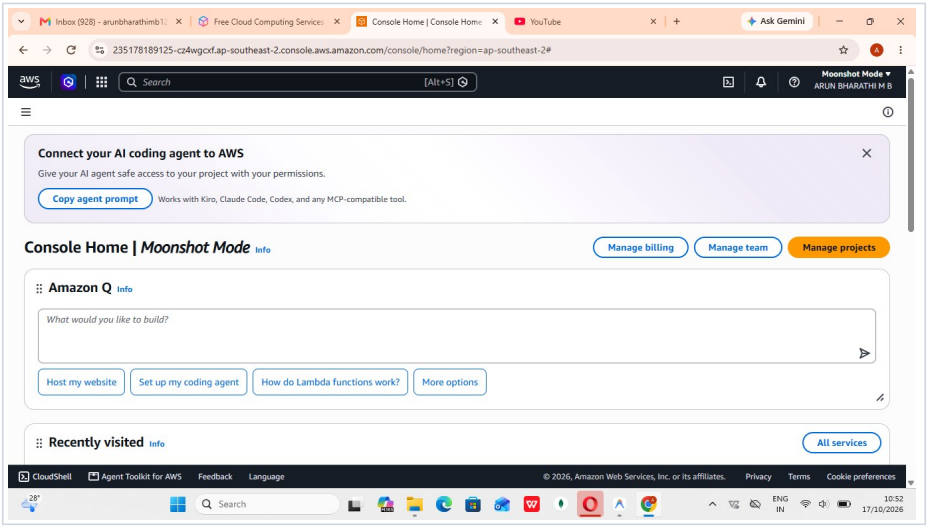



*Figure 5.1: AWS Management Console, set to the Sydney (ap-southeast-2) region.*

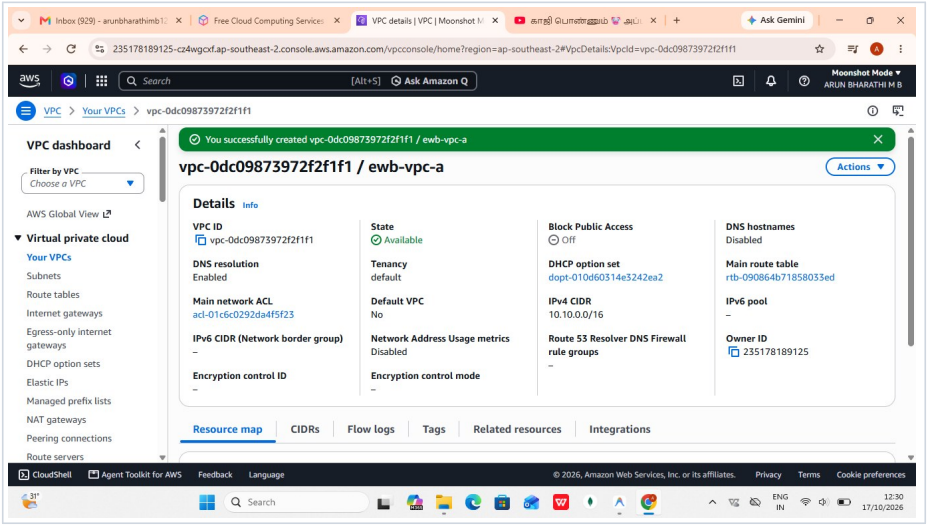



*Figure 5.2: ewb-vpc-a (vpc-0dc09873972f21f1) created and in the Available state.*

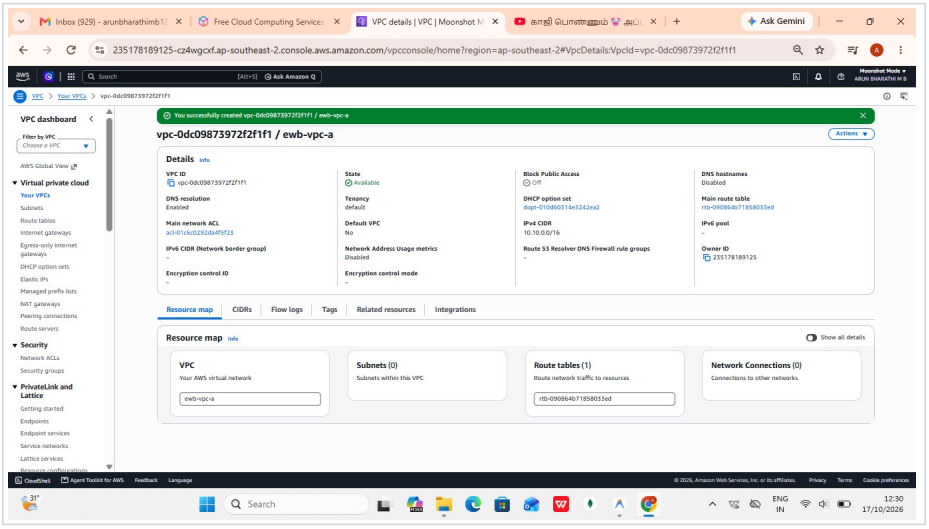



*Figure 5.3: Resource map of VPC A right after creation, before the subnets were added.*

> **Why this matters**
> The resource map shows 0 subnets and one route table. That one table is the VPC's automatically created main route table. I did not rely on it; I created custom tables for each subnet so that the routing of every subnet is explicit.

## 5.2 Internet gateway and route tables for VPC A

I created an internet gateway, `ewb-igw-a`, and attached it to VPC A. Then I made a custom route table, `ewb-rt-a-public`, linked it explicitly to `ewb-a-public` and added a default route (0.0.0.0/0) pointing at the internet gateway. The private subnet got its own table, `ewb-rt-a-private`, which only has the local route for now.

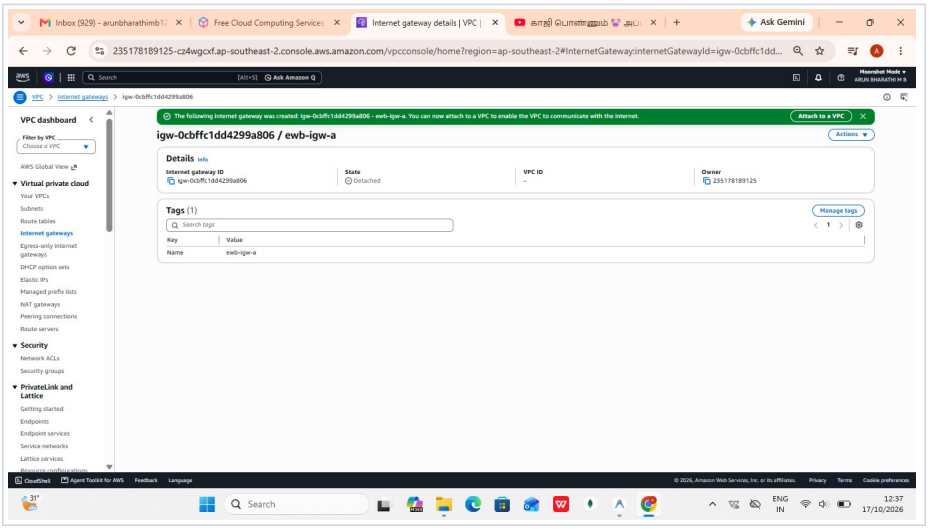



*Figure 5.4: ewb-igw-a (igw-0cbffc1dd4299a806) created; its state is still Detached.*

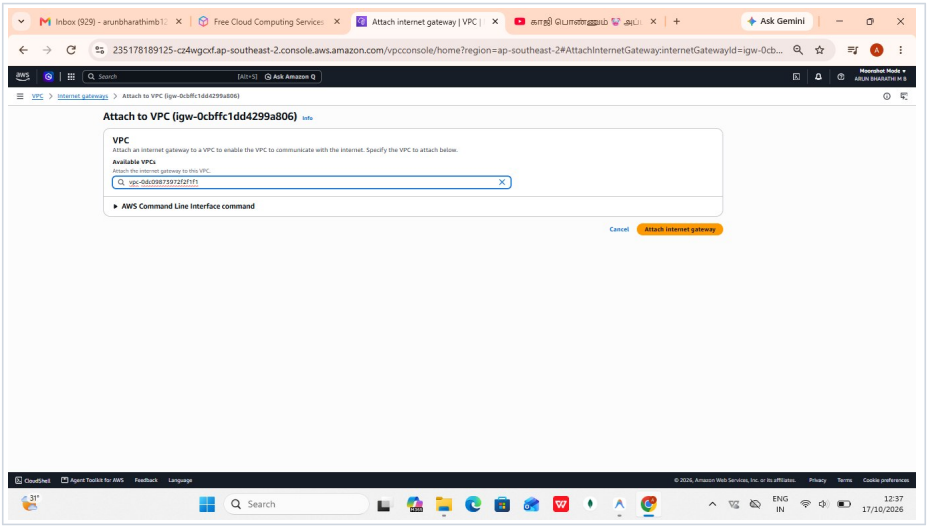



*Figure 5.5: Attaching ewb-igw-a to VPC A.*

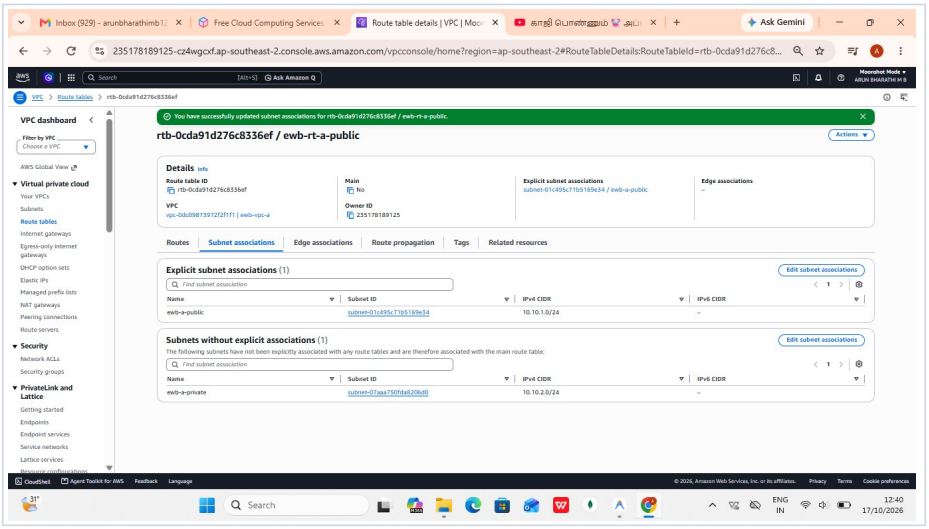



*Figure 5.6: ewb-rt-a-public explicitly associated with the subnet ewb-a-public.*

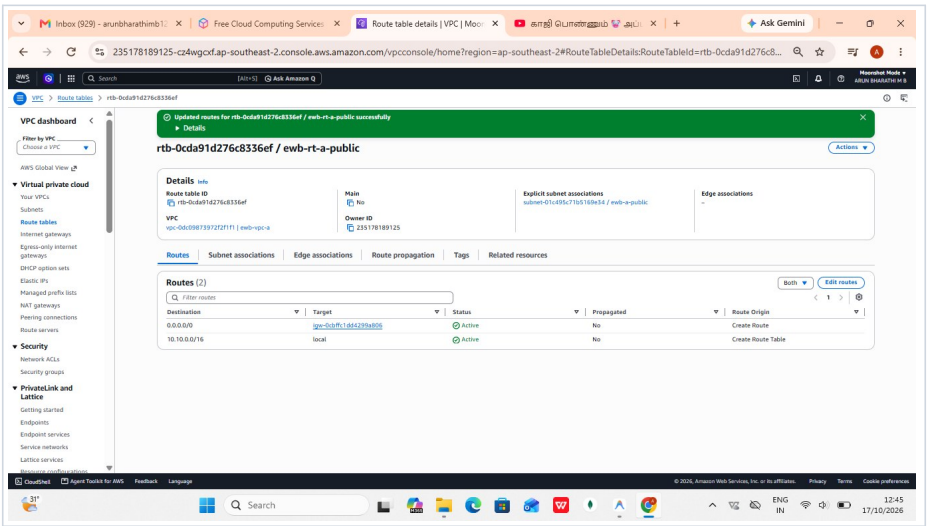



*Figure 5.7: Routes in ewb-rt-a-public: the default route to the internet gateway is Active, plus the local route.*

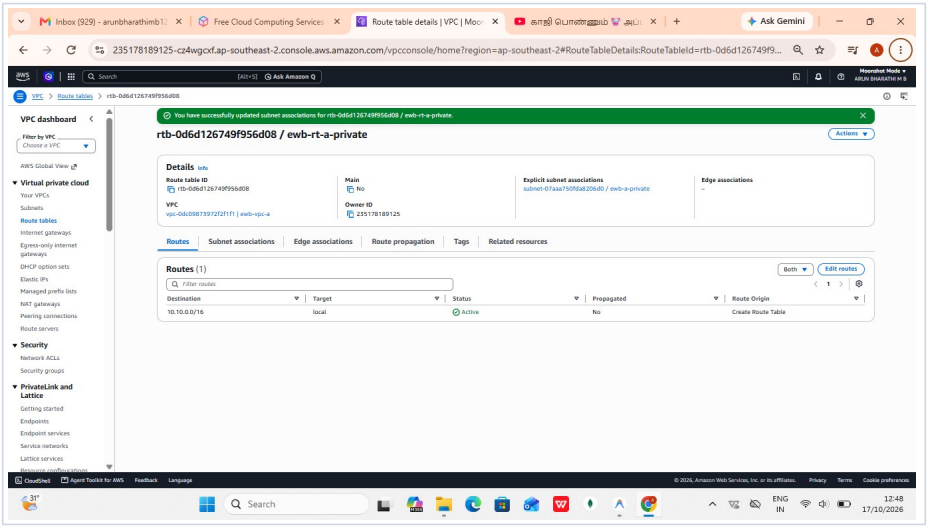



*Figure 5.8: ewb-rt-a-private associated with ewb-a-private; only the local route 10.10.0.0/16 is present.*

> **Why this matters**
> A gateway that is created but not attached does nothing, which is why Figure 5.4 shows it as Detached. Attaching it is a separate step. Likewise, the route to the gateway only takes effect for subnets the table is associated with, so the association in Figure 5.6 is what actually makes ewb-a-public public.

## 5.3 VPC B and its private subnet

VPC B uses `10.20.0.0/16` and holds a single private subnet, `ewb-b-private` (10.20.1.0/24). I deliberately gave it no internet gateway, so nothing can reach it or leave it directly over the internet. It has its own route table, `ewb-rt-b-private`, associated with that subnet.

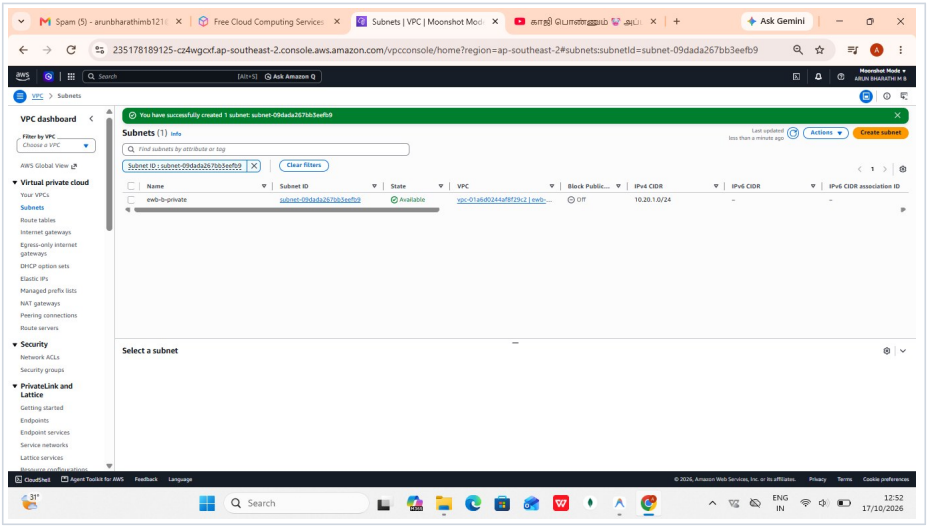



*Figure 5.9: Subnet ewb-b-private (subnet-09dada267bb3eefb9) created and Available in VPC B.*

## 5.4 Security groups

I kept the firewall rules as tight as I could by using single /32 addresses. The bastion accepts SSH only from my own IP. The two private app instances accept SSH only from the bastion's private IP (`10.10.1.10/32`) and ICMP echo requests only from the peer instance. The full matrix is in section 4.2.

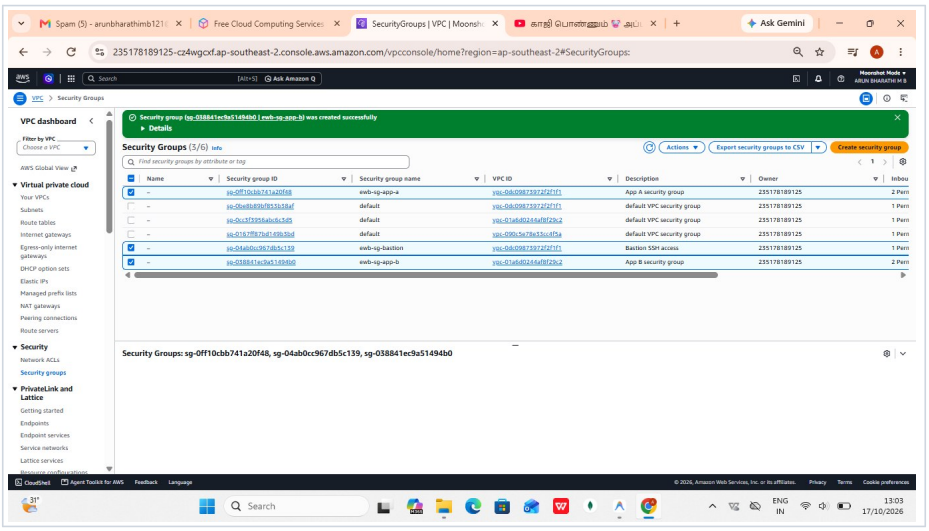



*Figure 5.10: Security groups created for the project: ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b.*

## 5.5 EC2 instances

I launched three Amazon Linux 2023 `t3.micro` instances with fixed private IPs: `ewb-bastion` (10.10.1.10), `ewb-app-a` (10.10.2.10) and `ewb-app-b` (10.20.1.10). Neither app instance has a public IP.

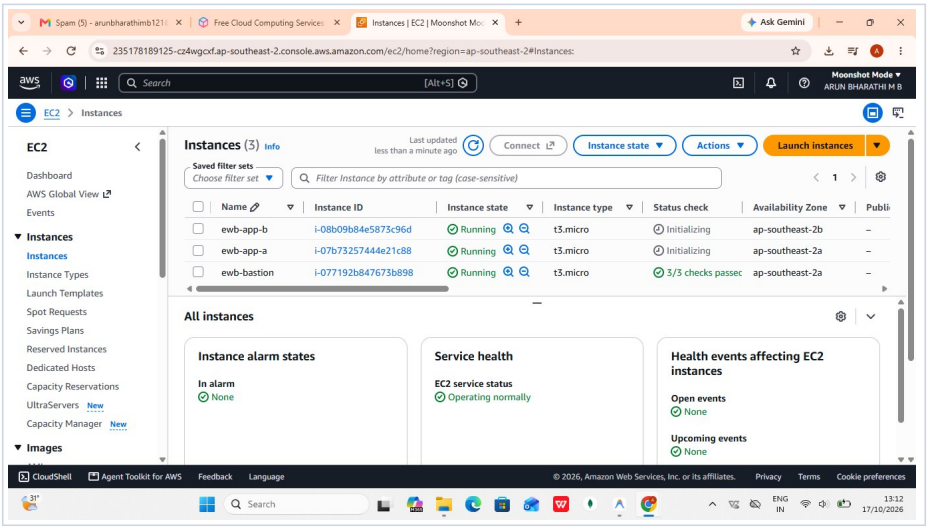



*Figure 5.11: All three instances running (the two app instances were still finishing their status checks).*

The bastion is the only one of the three that shows the full "3/3 checks passed" in this screenshot; the two app instances were still initializing when it was taken, which is normal for instances that have just launched. The instances sit in different availability zones: App-B is in ap-southeast-2b and the other two are in ap-southeast-2a.

## 5.6 The peering connection and the return routes

I created the peering connection `ewb-a-b-peer` with `vpc-0dc09873972f21f1` as requester and `vpc-01a6d0244af8f29c2` as accepter. Once I accepted the request, `pcx-0273ef24e21641107` changed to Active. A peering connection does nothing by itself, so I then added a route on each side that points the other VPC's range at it.

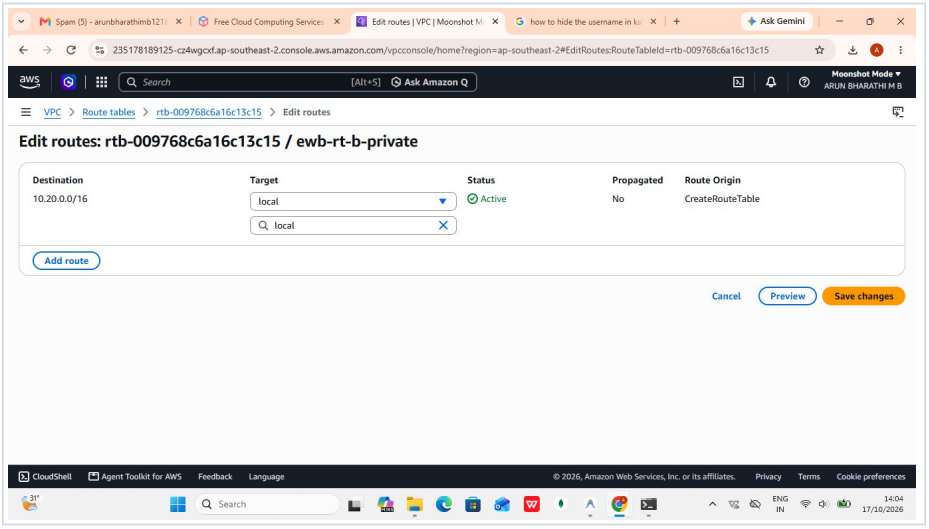



*Figure 5.12: Editing ewb-rt-b-private. At this point it only has the local route 10.20.0.0/16.*

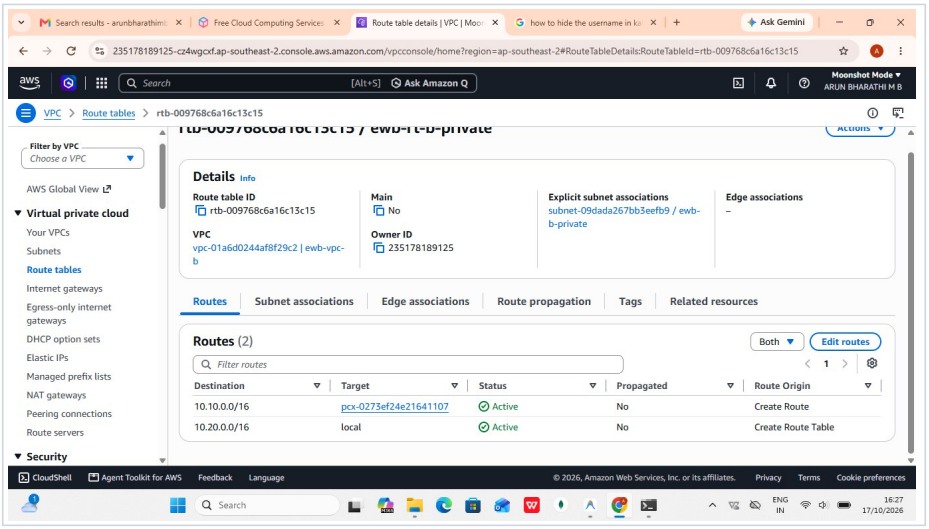



*Figure 5.13: ewb-rt-b-private after the update, with 10.10.0.0/16 now pointing at pcx-0273ef24e21641107.*

> **Why this matters**
> Figure 5.12 is the "before" and Figure 5.13 is the "after" for VPC B. The one new line is the return route. Without it, App-B would receive the ping but its reply would have nowhere to go. The matching route on VPC A's side points 10.20.0.0/16 at the same peering ID.

# 6. Testing and Results

I tested in a deliberate order: first prove there is no connectivity, then add the missing pieces, then prove there is. That way a successful ping can be credited to the peering setup and nothing else.

## 6.1 Before peering (expected to fail)

Before adding any peering routes, I pinged `ewb-app-b` (10.20.1.10) from `ewb-app-a`. All packets were lost, which confirmed that the two VPCs were fully isolated at that point.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3152ms
```

There was no route to 10.20.0.0/16 in VPC A, so the packets had nowhere to go and were silently dropped. A silent timeout like this is typical of a missing route or a blocking firewall, not of a refused connection.

### 6.2 After peering: App-A to App-B

Once the peering connection was active and the routes were in place, I ran the same ping again from App-A. All four packets came back, with an average round-trip time of **0.776 ms** (min 0.661 ms, max 1.046 ms).

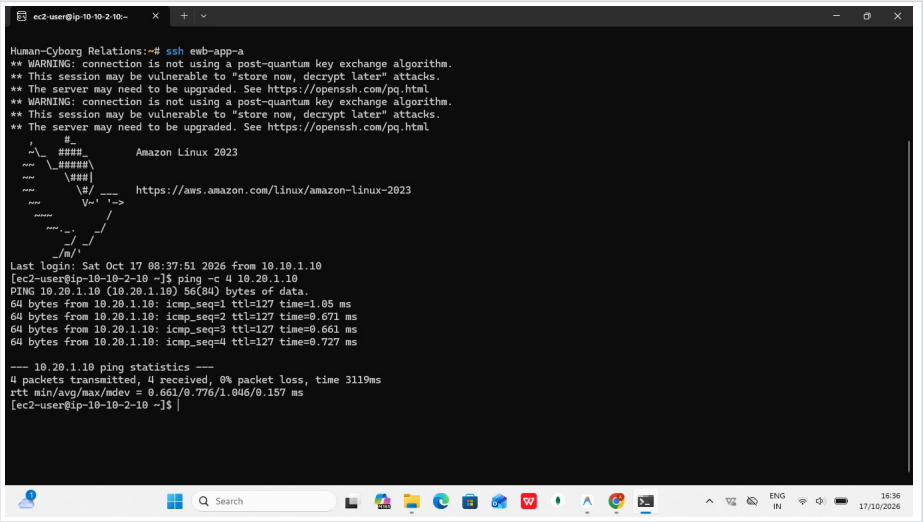



*Figure 6.1: Terminal session. I SSH'd into ewb-app-a through the bastion and pinged 10.20.1.10 across the peering link: 4 sent, 4 received, 0% loss.*

Sub-millisecond latency is what I would expect for two instances inside the same region, since the traffic stays on AWS's network and never leaves for the internet.

### 6.3 ProxyJump SSH and the reverse direction

To check administrative access, I connected from my laptop straight to the private instances with OpenSSH ProxyJump through the bastion. The "Last login" line in Figure 6.1 shows the session arriving from 10.10.1.10, which is the bastion. From App-B I also pinged App-A (10.10.2.10) and got 0% packet loss, so the connection works in both directions.

### 6.4 Summary of results

| Test | Condition | Expected | Result |
|---|---|---|---|
| App-A to App-B ping | Peering routes not yet added | Fail | 100% packet loss |
| App-A to App-B ping | Peering active, routes on both sides | Succeed | 0% loss, average 0.776 ms |
| App-B to App-A ping | Peering active, routes on both sides | Succeed | 0% packet loss |
| SSH from laptop to private instances | ProxyJump via the bastion | Succeed | Connected; login shown from 10.10.1.10 |
| App-A to App-B ping | ICMP rule removed from ewb-sg-app-b | Fail | 100% packet loss |
| App-A to App-B ping | ICMP rule restored | Succeed | 0% packet loss |

## 7. Fault Injection Experiment

I wanted to see for myself that a route alone is not enough and that the security group really is doing its job. This is what I did:

1. I temporarily deleted the inbound ICMP echo rule (source 10.10.2.10/32) from `ewb-sg-app-b`.
2. I ran `ping -c 4 10.20.1.10` from App-A again. It dropped to 100% packet loss straight away, even though the route table entry was still there.
3. I put the rule back, and pings went through again with 0% loss.

Security groups are stateful, so I only needed the inbound rule for the request. The reply is allowed back automatically.

### 7.1 What the experiment shows

- **Routing and filtering are separate layers.** The route to VPC B stayed in place the whole time. Removing one firewall rule was enough to break the ping.
- **The change took effect immediately.** Security group edits apply to running instances right away, with no restart needed.
- **The failure looks the same as a missing route.** Both give a plain timeout, so when ping fails you have to check routes and security groups one at a time (see section 8).
- **Least privilege works.** Because the rule names one exact source, no other host in VPC A could have pinged App-B even though the network path exists.

## 8. Troubleshooting Guide

Based on what I learned in this project, this is the order I would follow if a connection between the two VPCs failed. The causes are the ones most commonly seen with peering setups.

| Symptom | Likely cause | What to check |
|---|---|---|
| Ping times out from App-A to App-B | No route for the other VPC's range | Open the source subnet's route table and confirm the destination CIDR points at the pcx ID. |
| Request arrives but no reply | Missing return route in the other VPC | Check the route table of the destination subnet for a route back to the source CIDR. |
| Route exists but ping still fails | Security group blocks ICMP | Confirm an inbound ICMP echo rule exists on the destination, with the correct /32 source. |
| SSH to a private instance times out | Rule only allows the wrong source | The app security group must allow port 22 from the bastion's private IP, not from your laptop. |
| Peering stays "Pending acceptance" | Accepter has not accepted | Accept the request from the accepter VPC's side. |
| Cannot create the peering connection | Overlapping CIDR blocks | Compare the two VPC ranges; they must not overlap. |
| Route entry shows "Blackhole" | Peering deleted or not Active | Check the peering connection status and recreate the route. |
| VPC B instance cannot reach the internet | Expected behaviour | Peering is non-transitive and VPC B has no IGW or NAT of its own. |
| Bastion unreachable from laptop | Admin IP changed or public IP missing | Update the /32 rule to the current IP and confirm the instance has a public address. |

> **Rule of thumb**
> Work through the layers in order: peering status, route table on the way out, route table on the way back, security group on the destination, then network ACLs. Checking them in that order finds most problems in a couple of minutes.

# 9. Security Considerations, Limitations and Improvements

## 9.1 What the design already does well

- The backend instances have no public IPs and no direct route to the internet.
- VPC B has no internet gateway at all, so exposure through it is not possible.
- Firewall rules use single-host /32 sources instead of wide ranges.
- Administration goes through one bastion, and its SSH port is open only to my own IP.
- Peering carries only the traffic that the route tables and security groups allow.

## 9.2 Limitations of this lab setup

- **Single bastion, single availability zone for VPC A.** If that zone or host failed, administration would stop. This is acceptable for a lab but not for production.
- **No outbound internet for private instances.** Without a NAT gateway or VPC endpoints, the private instances cannot download OS updates or packages. They also cannot install anything new.
- **Default network ACLs.** Subnets rely only on security groups for filtering.
- **No traffic logging.** I did not enable VPC flow logs, so there is no record of accepted or rejected traffic.
- **Fixed /32 rules depend on fixed IPs.** If an instance were replaced and got a new address, its rules would need updating. The administrator's home IP can also change.

## 9.3 What I would add next

| Improvement | What it solves |
|---|---|
| AWS Systems Manager Session Manager | Shell access to private instances with no bastion, no open SSH port and no public IP. |
| Security group references instead of /32 rules | Rules follow instances even when private IPs change. |
| VPC flow logs | A record of accepted and rejected traffic for auditing and troubleshooting. |
| NAT gateway or VPC endpoints | Lets private instances fetch updates without being reachable from the internet. |
| Subnets in a second availability zone | Removes the single-zone dependency for the bastion and app tiers. |
| Custom network ACLs | A second, stateless filtering layer around each subnet. |
| Transit Gateway | Replaces many peering links when more than a few VPCs need to talk to each other. |

# 10. Viva Questions and Answers

**Q1. Why does VPC B need its own return route?**

Routing is decided separately for each direction. VPC A's route table sends traffic for 10.20.0.0/16 over the peering connection, so the ping request reaches App-B. App-B then has to send an echo reply to 10.10.2.10, and the VPC B router has no idea where that address lives unless `ewb-rt-b-private` has a route for 10.10.0.0/16 via `pcx-0273ef24e21641107`. Without it the reply is dropped.

**Q2. Why does App-B's firewall see the bastion's private IP and not my laptop's public IP?**

With ProxyJump my laptop only opens an encrypted connection to the bastion. The bastion's SSH daemon then opens a brand-new TCP connection from its own private address (10.10.1.10) to App-B on port 22. So from App-B's side, the source is the bastion, and that is what the security group has to allow.

**Q3. Why is an Active peering connection not enough for traffic to flow?**

Active only means AWS has set up the link between the two VPCs. For traffic to actually pass, three more things have to line up: (1) route table entries that point at the peering ID, (2) security group rules that allow the protocol and port, and (3) network ACLs that allow the traffic in both directions.

**Q4. Can VPC B reach the internet through VPC A's internet gateway over the peering link?**

No. VPC peering is non-transitive. Traffic that arrives over a peering connection cannot be forwarded out through the other VPC's internet gateway, NAT gateway or virtual private gateway.

**Q5. What is the difference between a security group and a network ACL?**

A security group is attached to an instance's network interface, is stateful and only has allow rules. A network ACL is attached to a subnet, is stateless, and can both allow and deny, with rules evaluated in number order. Because an ACL is stateless, it needs rules for both directions of a conversation, whereas a security group automatically lets replies through.

**Q6. Why must the two VPC CIDR ranges not overlap?**

Each VPC's router needs an unambiguous route for the other VPC's range. If both used the same range, an address like 10.0.1.5 could mean either VPC, and the route table could not decide between the local route and the peering route. AWS refuses to create the peering connection in that case.

**Q7. Why use a bastion and ProxyJump instead of giving the app instances public IPs?**

Public IPs on the app instances would expose them to the internet and multiply the number of places that must be secured. With a bastion there is one hardened entry point, and the private instances only accept SSH from it. ProxyJump keeps the experience simple and means no private key has to be stored on the bastion.

**Q8. What would change if the bastion's private IP changed?**

The app security groups trust `10.10.1.10/32` specifically, so SSH through the bastion would stop working until the rule was updated. That is why I assigned fixed private IPs. A more flexible option is to let the app security groups reference the bastion's security group instead of an IP.

**Q9. When would you choose peering, a Transit Gateway or a VPN?**

Peering is the simplest and cheapest choice for connecting a small number of VPCs directly. A Transit Gateway is better when many VPCs or on-premises networks must connect, because peering needs a link for every pair and is not transitive. A VPN is used when one side is outside AWS, for example an office network.

**Q10. Does peering cost anything, and does it encrypt my traffic?**

There is no hourly charge for the peering connection itself, but data transferred between availability zones is billed at the standard inter-AZ rate, and App-A and App-B are in different zones. Peering is not a replacement for application-level encryption, so SSH or TLS should still be used for anything sensitive.

# 11. Cost Control and Cleanup

To stay inside the Free Tier and avoid surprise charges, I did two things once testing was done:

- Released the Elastic IP right away so it would not sit idle and get billed.
- Stopped all three EC2 instances while I wrote this report, so they use no instance hours.

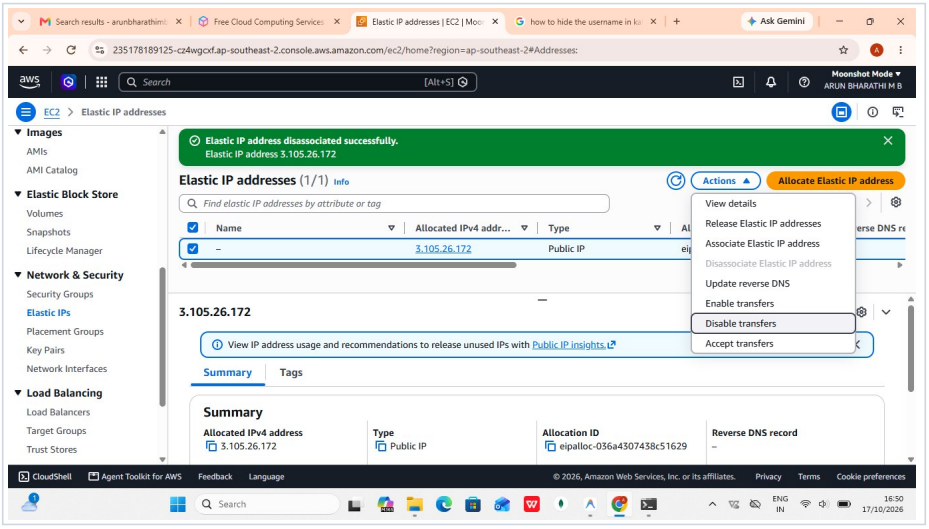



*Figure 11.1: Releasing Elastic IP 3.105.26.172 from the account.*

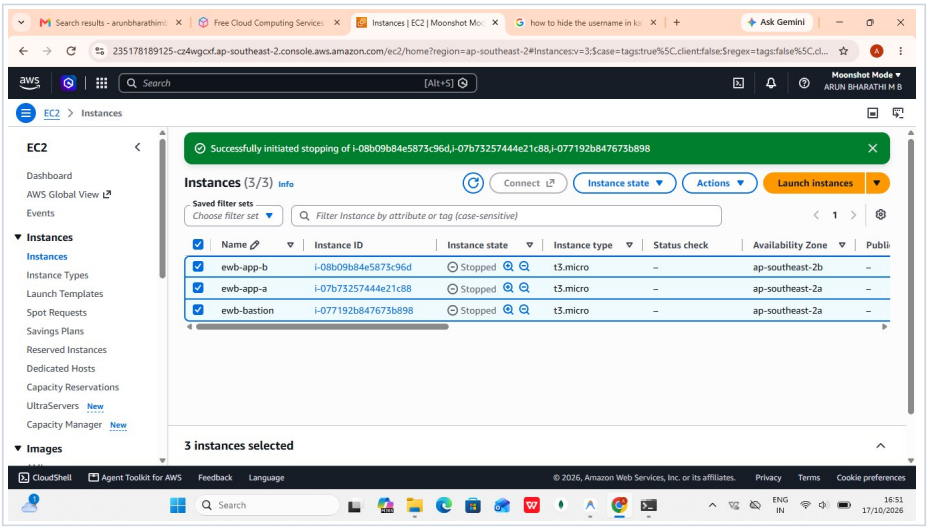



*Figure 11.2: All three EC2 instances in the Stopped state until the demo.*

## 11.1 Other things that can cost money

- **Public IPv4 addresses.** AWS charges for public IPv4 addresses, including ones attached to running instances, beyond the Free Tier allowance. Check the current AWS pricing page for the exact rate.
- **Stopped instances still have storage.** Their EBS volumes remain and are billed until the instances are terminated.
- **NAT gateways** bill hourly plus per GB, which is one reason this design uses peering instead.
- **Inter-AZ data transfer.** Small for ping tests, but it grows with real workloads.

## 11.2 Full cleanup order

When the project is finished and the demo is over, deleting everything in the right order avoids dependency errors:

1. Terminate the three EC2 instances (this also removes their volumes by default).
2. Delete the peering connection, then remove the routes that pointed at it.
3. Detach and delete the internet gateway.
4. Delete the custom route tables and the subnets.

5. Delete the security groups, then both VPCs.

# 12. Conclusion and Lessons Learned

This project gave me hands-on practice designing, building and testing a two-VPC setup on AWS. The main things I took away:

- **Network design:** planning CIDR blocks that do not overlap and separating public and private subnets.
- **Keeping backends private:** the app instances have no public IPs, and the bastion with SSH ProxyJump is the only way in.
- **VPC peering:** connecting two VPCs directly, with no NAT device or third-party appliance, and remembering that routes are needed on both sides.
- **Security groups:** writing least-privilege rules and understanding how stateful connection tracking affects them.

## 12.1 Lessons I would remember

1. "Active" is only the first step. Reachability needs routes, security groups and network ACLs to all agree.
2. Always test the broken state first. The 100% loss result before peering made the later 0% loss result meaningful.
3. Check both directions. A one-way route gives a symptom that looks like a firewall problem.
4. Break one thing on purpose. The fault injection taught me more about security groups than reading the documentation did.
5. Clean up straight away. Releasing the Elastic IP and stopping the instances is part of the job, not an afterthought.

---

**Candidate declaration**

I confirm that I built, tested and documented this project myself on AWS, as part of the EWB Cloud Labs curriculum.

**ARUN BHARATHI M B**
Student / Cloud Engineering Intern

# Appendix A: Equivalent AWS CLI Commands

I built the project in the AWS Management Console, as the screenshots show. These are the equivalent AWS CLI commands, so the same setup can be repeated or scripted. IDs in angle brackets stand for values returned by the earlier commands, and the AMI and key pair names are placeholders.

## A.1 VPC A, subnets and internet gateway

```
aws ec2 create-vpc --cidr-block 10.10.0.0/16 --region ap-southeast-2 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=ewb-vpc-a}]'

aws ec2 create-subnet --vpc-id <vpc-a-id> --cidr-block 10.10.1.0/24 \
  --availability-zone ap-southeast-2a
aws ec2 create-subnet --vpc-id <vpc-a-id> --cidr-block 10.10.2.0/24 \
  --availability-zone ap-southeast-2a

aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id <igw-id> --vpc-id <vpc-a-id>
```

## A.2 Route tables for VPC A

```
aws ec2 create-route-table --vpc-id <vpc-a-id>          # public table
aws ec2 create-route --route-table-id <rtb-public> \
  --destination-cidr-block 0.0.0.0/0 --gateway-id <igw-id>
aws ec2 associate-route-table --route-table-id <rtb-public> --subnet-id <subnet-public>

aws ec2 create-route-table --vpc-id <vpc-a-id>          # private table
aws ec2 associate-route-table --route-table-id <rtb-private-a> --subnet-id <subnet-private-a>
```

## A.3 VPC B and its private route table

```
aws ec2 create-vpc --cidr-block 10.20.0.0/16
aws ec2 create-subnet --vpc-id <vpc-b-id> --cidr-block 10.20.1.0/24 \
  --availability-zone ap-southeast-2b
aws ec2 create-route-table --vpc-id <vpc-b-id>
aws ec2 associate-route-table --route-table-id <rtb-private-b> --subnet-id <subnet-private-b>
```

## A.4 Peering connection and routes on both sides

```
aws ec2 create-vpc-peering-connection --vpc-id <vpc-a-id> --peer-vpc-id <vpc-b-id>
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id <pcx-id>

aws ec2 create-route --route-table-id <rtb-private-a> \
  --destination-cidr-block 10.20.0.0/16 --vpc-peering-connection-id <pcx-id>
aws ec2 create-route --route-table-id <rtb-private-b> \
  --destination-cidr-block 10.10.0.0/16 --vpc-peering-connection-id <pcx-id>
```

## A.5 Security groups and instances

```
aws ec2 create-security-group --group-name ewb-sg-app-b \
  --description "App B security group" --vpc-id <vpc-b-id>

aws ec2 authorize-security-group-ingress --group-id <sg-app-b> \
  --protocol tcp --port 22 --cidr 10.10.1.10/32
aws ec2 authorize-security-group-ingress --group-id <sg-app-b> \
  --ip-permissions 'IpProtocol=icmp,FromPort=8,ToPort=-1,IpRanges=[{CidrIp=10.10.2.10/32}]'

aws ec2 run-instances --image-id <al2023-ami-id> --instance-type t3.micro \
  --subnet-id <subnet-private-b> --private-ip-address 10.20.1.10 \
  --security-group-ids <sg-app-b> --key-name <key-pair-name>
```

# Appendix B: SSH Configuration for ProxyJump

This is an example of the `~/.ssh/config` entries that make `ssh ewb-app-a` and `ssh ewb-app-b` work through the bastion. The public IP and key path are placeholders; use your own.

```
Host ewb-bastion
    HostName <bastion-public-ip>
    User ec2-user
    IdentityFile ~/.ssh/<your-key>.pem

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/<your-key>.pem
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/<your-key>.pem
    ProxyJump ewb-bastion
```

The same thing works without a config file by using the one-line form, for example `ssh -J ec2-user@ ec2-user@10.20.1.10`. Either way, the private key stays on my laptop and is never copied to the bastion.

# Appendix C: Glossary

| Term | Meaning |
|---|---|
| VPC | Virtual Private Cloud: an isolated virtual network in one AWS region. |
| CIDR | Notation for an IP range, such as 10.10.0.0/16, where the number after the slash is the count of fixed leading bits. |
| Subnet | A slice of a VPC range that lives in one availability zone. |
| Route table | A list of destination ranges and where to send traffic for each. |
| Internet gateway (IGW) | The VPC component that connects subnets to the public internet. |
| VPC peering | A direct, private network link between two VPCs. |
| pcx-… | The ID prefix AWS uses for a peering connection. |
| Security group | A stateful, instance-level firewall with allow rules only. |
| Network ACL | A stateless, subnet-level firewall with allow and deny rules. |
| Bastion host | A hardened server in a public subnet used as the single entry point for administration. |
| ProxyJump | An OpenSSH option that connects to a target host through an intermediate jump host. |
| ICMP | The protocol used by ping; "echo request" and "echo reply" are its two messages. |
| Stateful | Remembers a connection, so replies to allowed traffic are let through automatically. |
| Non-transitive | Traffic cannot pass through one peered VPC on its way to a third network. |
| Availability zone (AZ) | One of several isolated data-centre groups within an AWS region, for example ap-southeast-2a. |

# Appendix D: Final Verification Checklist

A quick list I used to confirm that every part of the project was in place:

| Check | Done |
|---|---|
| VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) created, ranges do not overlap | Yes |
| Internet gateway attached to VPC A only | Yes |
| ewb-rt-a-public has 0.0.0.0/0 to the IGW and is associated with ewb-a-public | Yes |

| Check | Done |
|---|---|
| Private subnets use their own route tables with no route to the IGW | Yes |
| App instances have no public IPs | Yes |
| Security groups use /32 sources only | Yes |
| Peering connection pcx-0273ef24e21641107 is Active | Yes |
| Route to 10.20.0.0/16 on the VPC A side and 10.10.0.0/16 on the VPC B side | Yes |
| Ping failed before peering and succeeded after it | Yes |
| Fault injection: removing the ICMP rule broke ping, restoring it fixed it | Yes |
| Elastic IP released and instances stopped after testing | Yes |